

Informe de cambios en core e infra

Comparacion tecnica entre Libreria_is y ProyectoIS

CORE	8 nuevos	9 modificados	28 sin cambios funcionales
INFRA	6 nuevos	0 equivalentes	Implementaciones concretas

Alcance

El documento identifica los archivos agregados y modificados para convertir la estructura base de ProyectoIS en un analizador funcional. Libreria_is no cuenta con historial Git; por tanto, el resultado corresponde a una comparacion directa del estado actual de ambos proyectos.

Cambios en core

1. core.application

Archivo nuevo: ProjectModelService.java

Es el servicio que coordina todo el proceso:

1. Solicita la extracción del proyecto.
2. Obtiene clasificadores y tipos pendientes.
3. Ejecuta los deductores de relaciones.
4. Devuelve un UmlModel completo.

Conecta estas abstracciones:

ProjectModelExtractor
NameScopeResolver
RelationshipDeductor
RelationshipResolver

2. core.application.context

Archivo nuevo: ResolutionKind.java

Agregué una clasificación explícita para cada relación pendiente:

TYPE_DEPENDENCY
ASSOCIATION
HIERARCHY
NESTING

Esto evita representar los tipos de relaciones mediante varias banderas booleanas difíciles de interpretar.

Modificado: PendingResolution.java

Antes utilizaba:

boolean isHierarchyClaim
boolean isNestingClaim

Ahora utiliza:

ResolutionKind kind

También agregué:

- Validación de kind.
- Limpieza de espacios.
- Eliminación de imports duplicados.
- Métodos derivados isHierarchyClaim() e isNestingClaim().

Esto permite diferenciar una asociación de una dependencia, algo que la estructura original no podía hacer correctamente.

Modificado: ResolutionContext.java

Agregué:

```
registerUnresolvedAssociation(...)  
registerUnresolvedNesting(...)
```

Con esto, el extractor puede registrar:

- Campos como asociaciones.
- Tipos internos como relaciones de anidamiento.

También se conservaron los métodos existentes para jerarquías y dependencias.

Modificado: UnresolvedTypeRegistry.java

Se cambió el registro para usar ResolutionKind.

Además, agregué:

```
registerAssociation(...)
```

Ahora el registro distingue entre:

- Tipos usados por métodos y parámetros: dependencia.
- Tipos usados como campos o componentes de records: asociación.
- Herencia e implementación: jerarquía.
- Tipos internos: anidamiento.

3. core.application.deduction

Estos cuatro archivos son nuevos.

AssociationRelationshipDeductor.java

Convierte los campos y componentes de records en:

```
UmlAssociation
```

Solo crea la relación cuando el tipo puede resolverse como otro clasificador interno.

HierarchyRelationshipDeductor.java

Detecta:

- Herencia con extends.
- Implementaciones con implements.

Produce:

```
UmlGeneralization
```

Y establece si la relación es una implementación de interfaz.

NestingRelationshipDeductor.java

Detecta clases, interfaces, enums, records y anotaciones declarados dentro de otro tipo.

Produce:

```
UmlNesting
```

También conserva si el tipo interno es static.

TypeDependencyDeductor.java

Detecta dependencias producidas por:

- Parámetros.

- Retornos.
- Constructores.
- Miembros de anotación.
- Tipos genéricos asociados a estos elementos.

Produce:

```
UmlDependency
```

4. core.application.resolver

Archivo nuevo: DefaultNameScopeResolver.java

Implementa realmente NameScopeResolver.

Resuelve un nombre como Curso buscando:

1. Nombre completamente calificado.
2. Tipo interno.
3. Tipo del mismo paquete.
4. Import explícito.
5. Import con *.
6. Coincidencia única por nombre simple.

También elimina temporalmente:

- Genéricos, como <Curso>.
- Dimensiones de arreglos, como Curso[].

5. core.domain.classifier

Modificado: UmlClassifier.java

Agregué validación para los parámetros genéricos:

```
templateParameters
```

Ahora:

- No pueden contener valores nulos.
- No pueden estar vacíos.
- Se limpian los espacios.

El resto de los cambios de este archivo son principalmente de formato y legibilidad.

6. core.domain.element

Modificado: UmlNamespace.java

Agregué:

```
fromQualifiedName(String qualifiedName)
```

Este método convierte:

```
com.universidad.modelo
```

en:

```
["com", "universidad", "modelo"]
```

El extractor de JavaParser lo usa para transformar los paquetes Java en namespaces del dominio.

7. core.domain.feature

Modificado: UmlOperation.java

Agregué:

- Limpieza del nombre con trim().
- Validación de estereotipos vacíos.
- Normalización del estereotipo.

Esto permite usar estereotipos internos como:

```
constructor
```

```
annotationMember
```

Modificado: UmlParameter.java

Agregué:

- Validación más clara.
- Limpieza del nombre del parámetro con trim().

Modificado: UmlProperty.java

Agregué limpieza del nombre de la propiedad con trim().

Los demás cambios son de formato y legibilidad.

8. core.domain.spec

Modificado: UmlClassification.java

Originalmente contenía:

```
CLASS
```

```
INTERFACE
```

```
ENUMERATION
```

Agregué:

```
RECORD
```

```
ANNOTATION
```

Esto fue necesario para que el inventario no tratara records y anotaciones como clases ordinarias.

9. core.ports

Archivo nuevo: ProjectModelExtractor.java

Define el contrato:

```
ResolutionContext extract(Path sourceDirectory)
```

Gracias a este puerto, core no depende directamente de JavaParser. La implementación concreta está en `infra.javaparser`.

Archivos de core copiados sin cambios funcionales

Estos archivos mantienen la estructura de ProyectoIS:

core.application.registry

- DeductionEvent.java
- DeductionStatus.java
- DiagnosticTracker.java
- DiagnosticTrackerImpl.java
- ExtractionSnapshot.java
- ExtractionSnapshotImpl.java
- ResolutionRegistry.java
- ResolutionRegistryImpl.java

core.application

- RelationshipDeductor.java
- NameScopeResolver.java
- RelationshipResolver.java

core.domain

- UmlElement.java
- UmlConcreteClassifier.java
- UmlModel.java
- UmlType.java
- RelationshipStrength.java
- UmlRelationship.java
- UmlAssociation.java
- UmlDependency.java
- UmlGeneralization.java
- UmlNesting.java
- AggregationKind.java
- UmlModifier.java
- UmlVisibility.java

core.ports

- DiagramWriter.java
- DomainPolicyProvider.java
- UmlGenStorageException.java

- UmlRenderer.java

Cambios en infra

Todo el contenido actual de infra fue agregado para proporcionar implementaciones concretas a los puertos de core.

1. infra.config

PropertiesDomainPolicyProvider.java

Implementa:

DomainPolicyProvider

Lee analisis.properties y administra:

- libraries.whitelist.
- libraries.blacklist.
- Tipos primitivos.
- Biblioteca estándar java.* y javax.*.
- Tipos estructurales permitidos o bloqueados.

2. infra.javaparser

JavaParserProjectModelExtractor.java

Es el adaptador principal de JavaParser.

Realiza lo siguiente:

- Recorre todos los archivos .java.
- Lee paquetes e imports.
- Respeta paquetes y clases excluidas.
- Registra clasificadores.
- Procesa tipos internos recursivamente.
- Detecta herencia e implementación.
- Detecta asociaciones por campos.
- Detecta dependencias por parámetros y retornos.
- Procesa arrays, genéricos, wildcards, uniones e intersecciones.
- Registra advertencias y errores de análisis.

JavaParserClassifierMapper.java

Convierte declaraciones JavaParser en UmlClassifier.

Mapea:

- Clases.
- Clases abstractas.
- Interfaces.
- Enums.

- Records.
- Anotaciones.
- Campos.
- Componentes de records.
- Constantes de enums.
- Métodos.
- Constructores.
- Constructores compactos.
- Miembros de anotaciones.
- Parámetros genéricos.

JavaParserSpecMapper.java

Convierte modificadores Java en especificaciones UML:

public -> PUBLIC
private -> PRIVATE
protected -> PROTECTED

sin valor -> PACKAGE

abstract -> ABSTRACT
static -> STATIC
final -> LEAF o READ_ONLY

JavaParserTypeMapper.java

Convierte tipos JavaParser en UmIType:

- Primitivos.
- Void.
- Arrays multidimensionales.
- Tipos genéricos.
- Parámetros de plantilla.
- Tipos todavía no resueltos.

3. infra.report

TextInventoryRenderer.java

Implementa:

UmlRenderer

Genera la sección textual de:

- Clasificadores.
- Cantidad de propiedades.
- Cantidad de operaciones.
- Relaciones detectadas.

Traduce las relaciones a:

HEREDA
IMPLEMENTA
CONTIENE
ASOCIA
DEPENDEN_DE

Impacto de la interfaz UI en core e infra

Resultado principal

La interfaz se agrego como una capa independiente en is.generador.ui. No se agregaron dependencias de Swing ni referencias hacia ui dentro de core o infra. La direccion de dependencias se mantiene desde ui hacia los servicios existentes.

Capa	Cambios por la interfaz	Uso desde UI
core	Ningun cambio directo y exclusivo para Swing. Conserva el modelo UML, los puertos, la resolucion y la deduccion de relaciones.	Entrega UmlModel, clasificadores, propiedades, operaciones y relaciones.
infra	Ningun componente visual fue agregado. Los adaptadores existentes continuan leyendo Java, configuracion y produciendo reportes.	Proporciona JavaParserProjectModelExtractor, PropertiesDomainPolicyProvider y TextInventoryRenderer.
ui	Paquete nuevo y aislado con modelos de presentacion, servicios Swing, tema y paneles.	Transforma los resultados del analisis en resumen, estructura, detalle, diagnosticos y exclusiones.

Flujo de integracion

InventoryFrame -> InventoryUiService -> SoyLaPuertaJava -> ProjectModelService -> adaptadores de infra

SoyLaPuertaJava se modifico fuera de core e infra para abrir InventoryApplication en modo grafico y conservar la salida por consola cuando el entorno es headless.

Archivos agregados en ui

- ui.app.InventoryApplication: inicia la ventana Swing.
- ui.model.InventoryViewModel y ClassifierFileDetails: preparan datos para visualizacion.
- ui.service.InventoryUiService y SourceDetailExtractor: coordinan analisis, configuracion y detalle por archivo.
- ui.theme.UiTheme: centraliza colores, tipografia y tarjetas.
- ui.view.InventoryFrame, SummaryPanel, DetailPanel y ExclusionsPanel: construyen las pestañas y paneles.

Exclusiones desde la interfaz

El panel Exclusiones permite agregar o quitar paquetes y clases desde la interfaz. Cada cambio se guarda de forma atomica en analisis.properties, conserva las propiedades de whitelist y blacklist y ejecuta nuevamente el analisis. InventoryUiService envia las colecciones actualizadas a los metodos publicos de SoyLaPuertaJava, que reutilizan los contratos de core y los adaptadores de infra. La exclusion automatica de com.universidad.AppGenerator se muestra protegida para evitar que el punto de entrada del analisis aparezca en el inventario. Esta funcionalidad se implemento en ui; no fue necesario modificar las clases de core ni de infra.

Regla arquitectonica conservada

core no conoce JavaParser ni Swing; infra conoce core pero no ui; ui consume la fachada y los resultados del analisis. Por tanto, la interfaz puede cambiar sin alterar las reglas centrales del dominio.